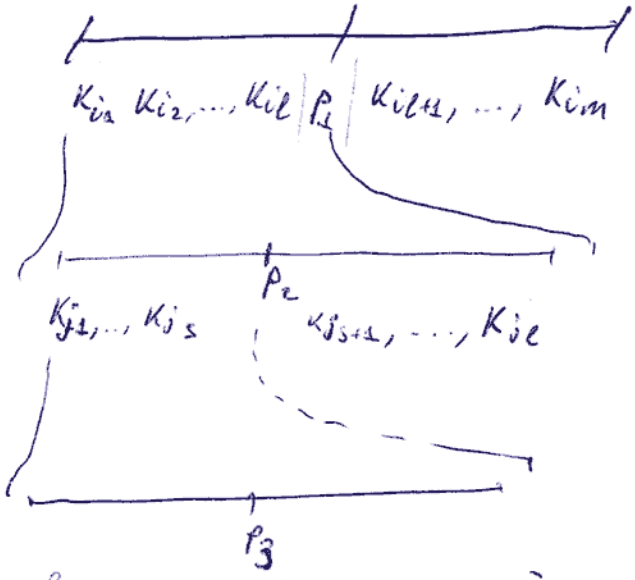
2-1. Будем действовать так:

1. Выбираем pivot
(например четвертое пятерок)

2. За время $O(n+m)$ делаем partition отдельно для n ключей и m запросов.
(Ключи разделяются по pivot-у, после чего известен размер левой части массива по которому делятся запросы).



3. Запускаем pivot + partition рекурсивно для всех частей где есть запросы. При условии что запросы разные, на каждой итерации для всех частей, куда попадают запросы мы будем тратить не больше чем $O(n+m)$:
Если c_i – куски поделенного массива, d_i - число попавших туда запросов. $\sum c_i \le n$ $\sum d_i = m$. Время pivot + partition внутри которого $O(\sum(c_i + d_i)) \asymp O(n+m)$.

Примерно за $\log_2 m$ итераций каждый из ключей получит «свой» кусок массива c_i. В каждом из них статистика ищется линейно $\Rightarrow$ за время $O(\sum c_i) = O(n)$ ищутся все статистики.

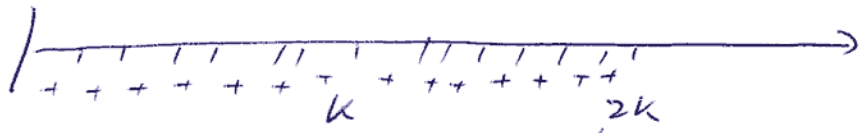
Итого $(n+n) \cdot \log_2 m + n \longrightarrow O((n+n)\log n)$

Ну, немного хуже, чем требуется в задаче...

2-2.

Будем поддерживать неупорядоченный массив размера $\leq 2K$. Действовать будем так:

1. Если размер массива $< 2K$, то вставки „достаточно" просто добавляются в конец.
2. При достижении размера $2K$ с помощью метода пятерок за время $O(2K)$ ищем медиану и еще за время $O(2K)$ с помощью нее оставляем только K элементов.



compact, $t = O(2K)$

Для n операций вставки суммарное время работы

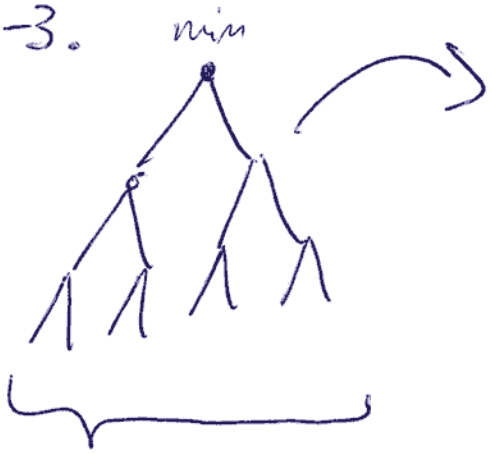
$$t = n + \left(\frac{n}{K}\right) \cdot 4K = 5n$$

n — число вставок в конец; $\left(\frac{n}{K}\right)$ — число „шагов"; $4K$ — сложность шагания.

Тогда, амортизированное время $\frac{t}{n} = 5 = O(1)$.

3. Get-Kth работает (с помощью того же метода пятерок) за время $O(2K) \simeq O(K)$ в худшем случае, что и требуется в задаче.

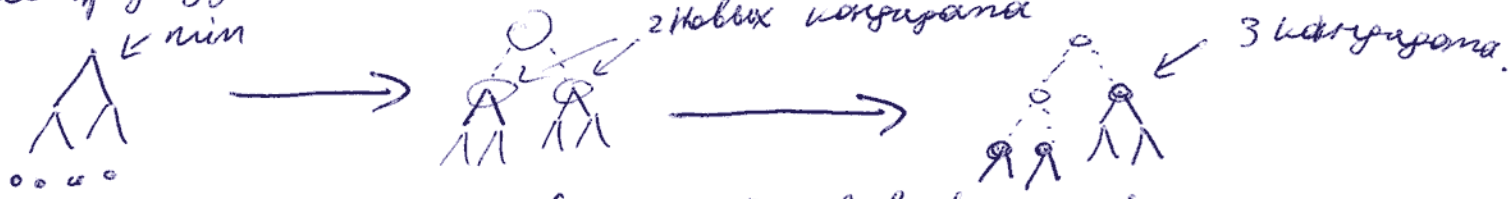
2-3.



– дополнительная куча.

n – изначальная куча

Можно заметить, что при нахождении очередного минимума следующие кандидаты на минимум это все предыдущие кандидаты (кроме найденного) + 2 его потомка.



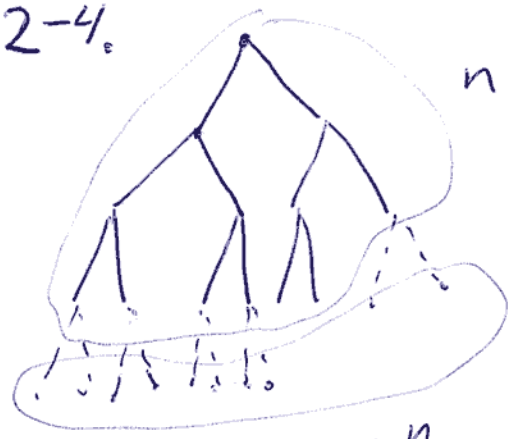
Используя этот факт будем действовать так:

1. Заведем вторую кучу. В ней будем хранить значения + указатели на вершины изначальной кучи.
2. В 0-й элемент во второй куче кладем из первой кучи.
3. На каждой итерации (Get Next Min) делаем так:
   1. Удаляем минимум из второй кучи за $t = \log(i)$, где i – ее размер.
   2. Добавляем двух потомков удаленного элемента за $2 \cdot \log(i)$. Размер кучи увеличился на 1.

Тогда, время работы $T = 3 \cdot \log(1) + 3 \cdot \log(2) + \dots + 3 \cdot \log(k)$

$= 3 \log(k!) \approx 3k \log k = O(k \log k)$

2-4.



1) Если $K > \frac{n}{2}$, то достаточно запустить обычный Heapify (Sift Down начиная с нижних вершин) он работает за $O(n+K) < O(3K) = O(K)$.

2) Если $K \leq \frac{n}{2}$, то будем запускать Sift Down только начиная с предков новых вершин. Остальная часть пути уже корректная. Это даст время

$$t = \frac{K}{2} \cdot 1 + \frac{K}{4} \cdot 2 + \frac{K}{8} \cdot 3 + \ldots \approx 2K = O(K).$$

Проблема, что такая оценка сойдется на не-корневой вершине:



вершина, на которой оценка $\frac{K}{2^i}$ сломается, т.к. она не корневая. Для нее нужно „доделать" работу высоты не более чем $\log n$:



, т.е. запустить Sift Down $\log n$ раз:

$$t = \log k + (1+\log k) + (2+\log k) + \ldots + \log(n-k) \leq \underbrace{\log 2n + \ldots + \log 2n}_{\log n} \leq$$

$$\leq \log^2 2n = O(\log^2 n)$$

Итого: $O(K + \log^2 n)$

3) Чтобы улучшить оценку, вместо того чтобы запускать Sift Down ровно $\log n$, $\log n$ раз, можно было бы сразу отсортировать весь путь этой вершины. Путь наверх состоит из $\log n$ вершин. Путь вниз ещё $\log K$. Но. Какие нижние вершины взять неизвестно. (Sift Down может идти в любую сторону...) Короче я не решил это, но явно можно хитро отсортировать целиком этот путь и получить $\log(n) \cdot \log(\log n)$... :)

2–5.

Структура: не более $\log n$ куч размеров $1, 3, 7, \ldots, 2^k - 1$

К примеру для:

$n = 1 \Rightarrow$ •

$n = 2 \Rightarrow$ • • (две кучи)

$n = 3 \Rightarrow$ 

$n = 4 \Rightarrow$ 

$n = 5 \Rightarrow$ 

$n = 6 \Rightarrow$ 

$n = 7 \Rightarrow$ 

$n = 8 \Rightarrow$ 

$n = 9 \Rightarrow$ 

$n = 10$ 

$n = 11$ 

$n = 12$ 

$n = 13$ 

$n = 14$ 

$n = 15$ 

То есть раскладываем n по слагаемым $2^k - 1$.

В каждый момент времени число куч одного размера не больше 2, а все остальные кучи убывают как степень 2.

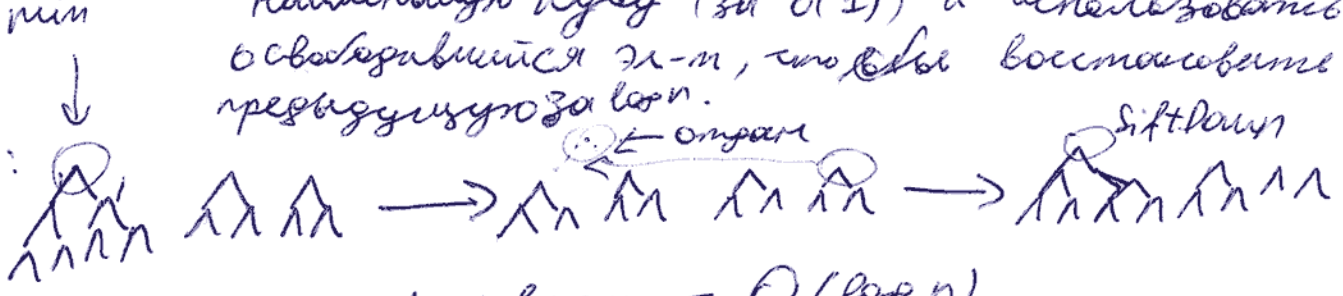
$\Rightarrow$ число куч $\approx \log n$.

ExtractMin: 1. За $\log(n)$ находим минимальный элемент среди $\log n$ корней куч.

2. За $O(1)$ разбираем кучу и отрываем корень.

3. Если структура куч нарушилась, ее надо восстановить (к примеру «разобрать») наименьшую кучу (за $O(1)$) и использовать освободившийся эл-т, чтобы восстановить предыдущую за $\log n$.

Пример: $n = 29$



Итого: $\log n + 1 + 1 + \log n = O(\log n)$

найти мин. разобрать SiftDown

Добавить элемент: элемент всегда добавляется
а) либо в новую кучу
б) либо, если есть 2 одинаковые кучи, каких Корень и запускается sift Down.

Посчитаем амортизированную сложность.
Пусть $n = 2^k - 1$. Тогда мы 1 раз делали снятие стоимостью k, 2 раза стоимостью $k-1$, 4 раза стоимостью $k-2$, ..., 2^{k-1} раз стоимостью 0.

Итого: $t = 2^0 \cdot k + 2^1 (k-1) + 4(k-2) + \dots + 2^{k-1} \cdot 1 \approx$

$\approx \sum 2^i \cdot k - \sum 2^i \cdot i = \left(k \cdot 2^k - k\right) - \left(k \cdot 2^k - 2^{k+1} + 2\right) \approx 2^{k+1}$

$\approx n = O(n)$.

амортизированно $\frac{n}{n} \to O(1)$.

2-6.

$h \in \mathcal{H}$, $h : X \to \{0, \dots, m\}$

$P(h(x_1) = y_1, \dots, h(x_k) = y_k) \le \frac{C}{m^k}$ } (C, k) – независимость

$h(x) = ((a_0 + a_1 x + \dots + a_{k-1} x^{k-1}) \bmod p) \bmod m$, $p > m$, p – простое

$\forall k \; \exists C : \forall m, p \; \{h(x)\}$ – (C, k) – независимые.

$P(h(x_1) = y_1, \dots, h(x_k) = y_k) \le \frac{C}{m^k}$ можно переписать как

$\max\limits_{\vec{x}, \vec{y}} \frac{\#\, h(\vec{x}) = \vec{y}}{|\mathcal{H}|} \le \frac{C}{m^k}$, где $h(\vec{x}) = \begin{pmatrix} h(x_1) \\ \vdots \\ h(x_k) \end{pmatrix}$

$|\mathcal{H}| = p^k$. (каждый коэфф. a_i принимает p возможных значений)

$\Rightarrow \max\limits_{\vec{x}, \vec{y}} \#\, h(\vec{x}) = \vec{y} \le C \left(\frac{p}{m}\right)^k$.

$h(\vec{x}) = \vec{y} \iff \begin{cases} (a_0 + a_1 x_1 + \dots + a_{k-1} x_1^{k-1}) \bmod p = y_1 + i_1 m \\ \dots \\ (a_0 + a_1 x_k + \dots + a_{k-1} x_k^{k-1}) \bmod p = y_k + i_k m \end{cases}$ –

система k ур. уравнений с k неизвестными $(a_0, \dots, a_{k-1})$ в поле F_p. Ее определитель – определитель Вандермонда

$|A| = \prod\limits_{1 \le i < j \le k} (x_j - x_i) \ne 0$ по условию, т.к. x_i – различны.

$\Rightarrow$ Для конкретных $\{i_1, \dots, i_k\}$ $\exists!$ решение. Сами $i_\ell \in \{0, \lfloor \frac{p}{m} \rfloor\}$,

т.е. всего наборов $|\{i_\ell\}| = \left(\lfloor \frac{p}{m} \rfloor + 1\right)^k$, таким образом $\#\, h(\vec{x}) = \vec{y} \le$

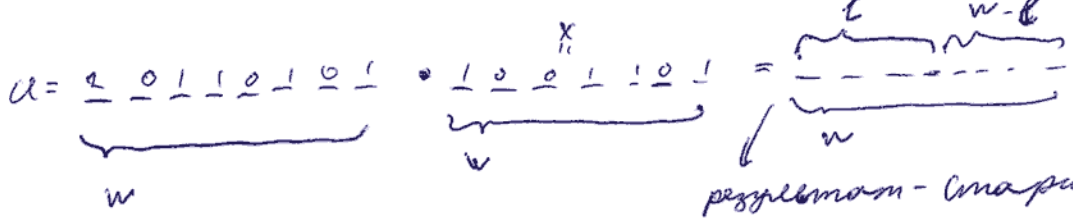
$\le \left(\lfloor \frac{p}{m} \rfloor + 1\right)^k \le 2^k \left(\frac{p}{m}\right)^k$. Взяв $C = 2^k$ получим, что $\#\, h(\vec{x}) = \vec{y} \le C \left(\frac{p}{m}\right)^k$

2-7.

$h \in \mathcal{H}$ - C-универсально, если $\forall x, y \; P(h(x) = h(y)) \le \frac{C}{m}$

$h: X \to \{0, \dots, m-1\}$

$X \in \{0, \dots, 2^w - 1\}$ $\quad h_a(X) = \lfloor ((ax) \bmod 2^w) / 2^{w-l} \rfloor$



$a \in \{0, \dots, 2^w - 1\}$
a - нечетно

Исходное условие переписывается как $\frac{\#(h(x) = h(y))}{|\mathcal{H}|} \le \frac{C}{2^l} \Leftrightarrow$

$\Leftrightarrow \#(h(x) = h(y)) \le C \frac{2^w / 2}{2^l} = C \cdot 2^{w-l-1}$

$h(x) = h(y) \Leftrightarrow (ax - ay) >> w - l = 0 \Leftrightarrow a(x-y) = b$, где старшие l битов b - нули.

Известно, что $az = b$ имеет! решение для a, если a, z - нечетные.

Пусть $(x-y) = z \cdot 2^q$, z - нечетно. $\Rightarrow$ $az \cdot 2^q = b \pmod{2^w}$

$\Rightarrow b = 2^q \cdot b_1$ (b_1 может быть и четным); $b =$ 

Тогда $az = b_1$ имеет! решение для a в арифметике $w-q$.

Всего решений в изначальной арифметике 2^q (т.к. старшие q бит для a перестают влиять на рез-тат из-за сдвига 2^q).

$\Rightarrow \forall x, y$ число решений уравнения $a(x-y) = b$ в 2^w арифметике равно $2^q \cdot 2^{w-l-q} \cdot \frac{1}{2} = 2^{w-l-1}$

2^q - четных $(x-y)$; 2^{w-l-q} - число подходящих b; $\frac{1}{2}$ - из них нечетных b_1 (т.к. a-нечет.)

$\Rightarrow \#(h(x) = h(y)) = 2^{w-l-1} \Rightarrow C \simeq 1.$